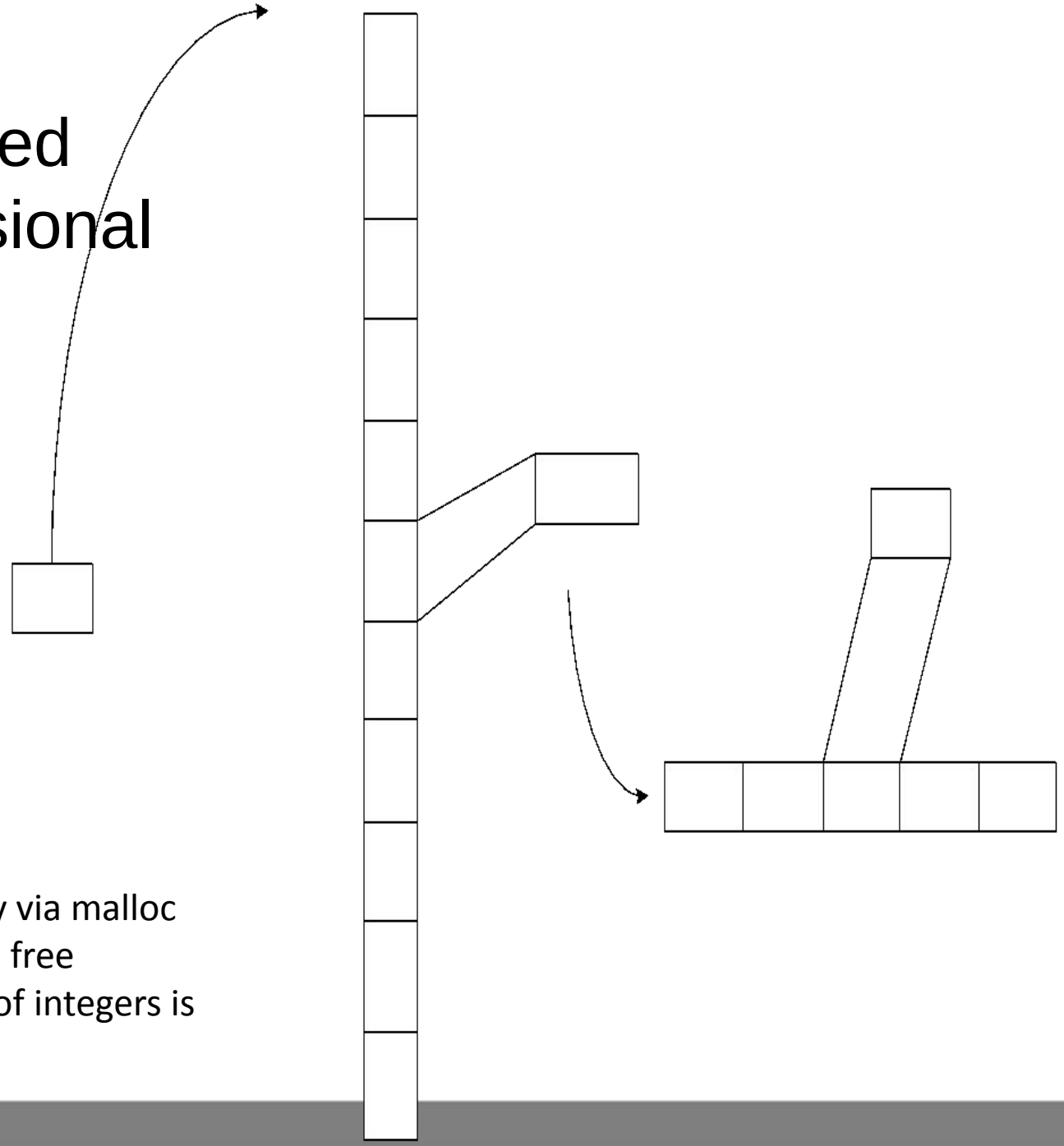
CS 2421

Heap Based Multidimensional Arrays

Heap-Based Multi-Dimensional Array



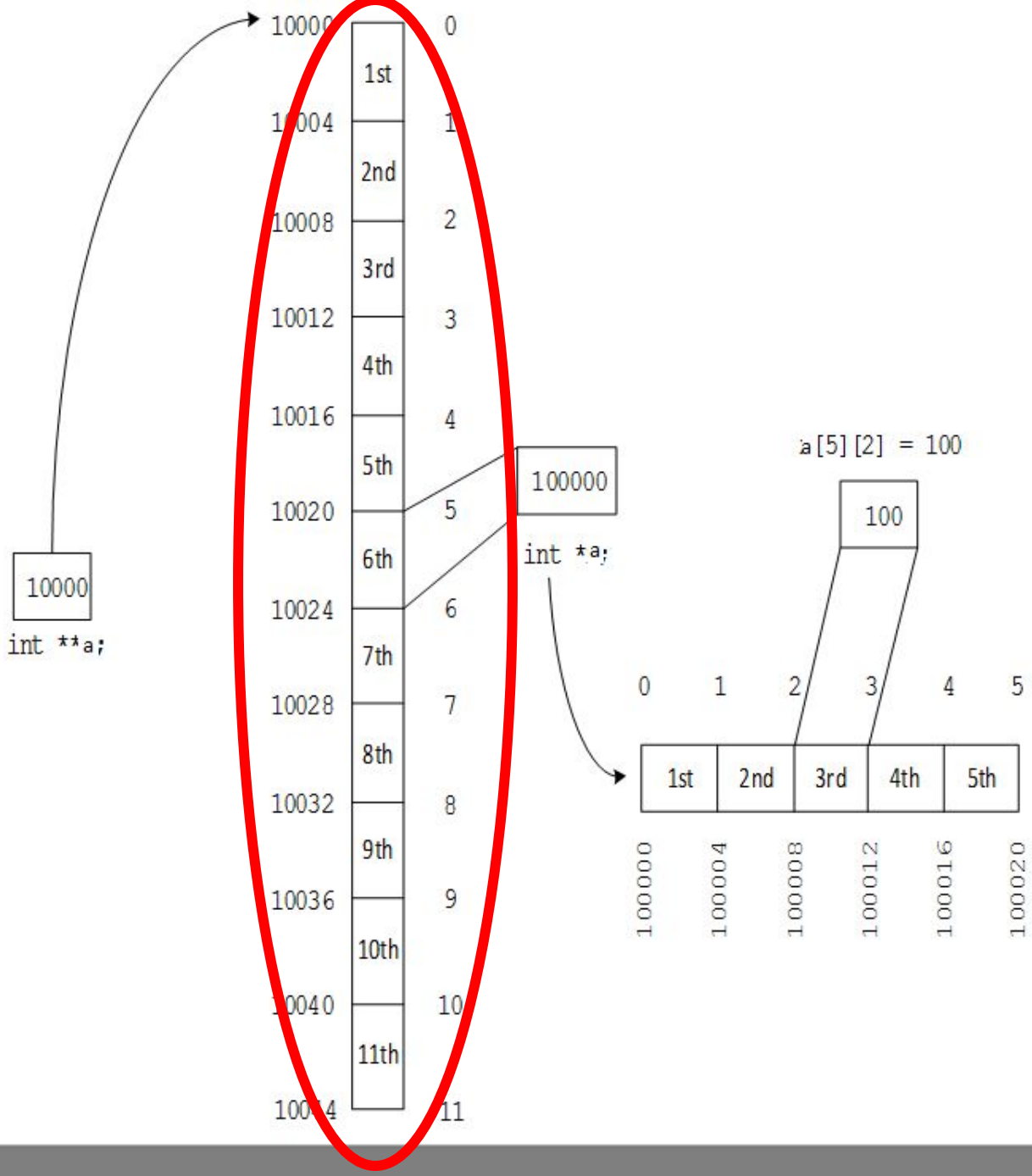
Must be allocated iteratively via malloc
Must be freed iteratively via free
Total storage of a 3x5 array of integers is (3x(5+1)) * 4

Heap Multi-dimensional array ex

```
int main()
{
      int i, m = 5;
      int **a =malloc(m*sizeof(int *));

      //Initialize the arrays
      for (i = 0; i < m; i++) {
          a[i]=malloc(n * sizeof(int));
      }
   // do_fun_array_stuff(a);
      // i_dont_like_this_value(a[3][4]);
   free(a);
      //This doesn't properly free everything
      //how do I free it completely?
      return 0;
}
```

Heap-Based Multi-Dimensional Array

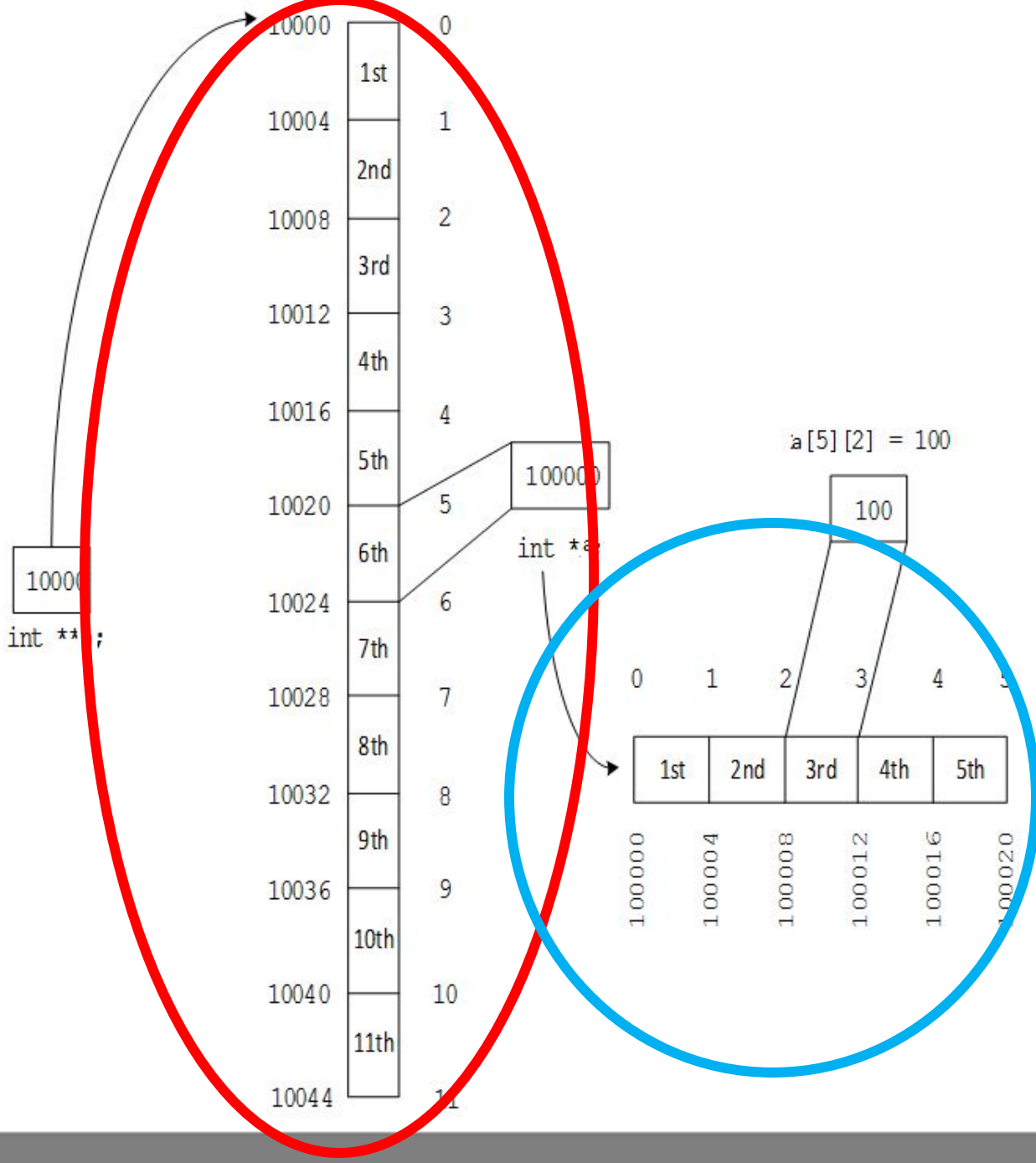


free(a) only frees the **first level** of the array

Heap Multi-dimensional array ex

```
void freeArray(int **a, int m) {
    int i;
    for (i = 0; i < m; ++i) {
        free(a[i]);
    }
    free(a);
}
// What is not possible with this
example?
```

Heap-Based Multi-Dimensional Array



free(a) only frees the **first level** of the array

Iterating over the first level and freeing them will free the **second level**

Character Arrays i.e. Strings

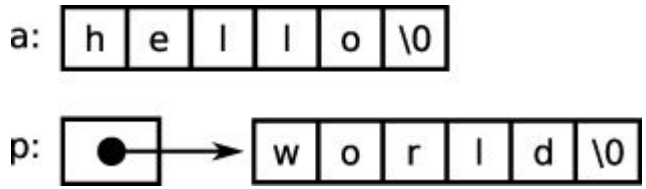
- Declarations:
  - char arr[] = {'c','o','d','e','\0'};
    - The null byte is required as a terminating byte when string is read as a whole.
  - char arr[] = "code";
    - Implies that there are 4 characters along with the NUL byte (i.e. the \0 character) so a “length” of 5.
- This type of array allocation, where the size of the array is determined at compile-time, is called static allocation.
- What about this declaration…
  - char *ch = “code”;
  - Implied constant… can’t change it

Pointers and Strings

- A string is an array of characters.
  - No string pointers in C.
  - When we point a pointer to a string, by default it holds the address of the first character of the string (just like an array)
- Ex: memory address without a reference operator(&)
  - char *ptr;
  - char str[40];
  - ptr = str;
- strlen(str) function defined in string.h
  - The strlen function returns the byte length of the null-terminated string
    - char str[32] = "hello, world"; //in memory ‘hello, world\0’
    - sizeof (str) ⇒ 32
    - strlen (str) ⇒ 12

Arrays vs. Pointers

```
char a[] = "hello";
char *p = "world";
```



Accessing is equivalent, but the machine code behind it behaves differently

```
a[3]
p[3]
```

Arrays vs. Pointers

```
char a[] = "string literal";
char *p = "string literal";
```

a[] is an array with the initial value of “string literal”. values may be altered. ex a[0] = ‘k’;

*p is a constant array of characters and can not be altered. EX: p[0] = ‘k’ will crash

sizeof Example

```
#include <stdio.h>
int test_sizeof(int buffer[]) {
    printf("sizeof buffer = %d\n", sizeof(buffer));
}
int main (int argc, char *argv[]) {
    int buffer[10];
    int i;
    for (i = 0; i < 10; i++)
        buffer[i] = i+i;
    printf("sizeof buffer = %d\n", sizeof(buffer));
    test_sizeof(buffer);
    return 0;
}
```

Output:
sizeof buffer = 40 (10 * int)
sizeof buffer = 4 (int *)

sizeof Example

- What happened?
- sizeof is evaluated at compile time, not at runtime. In the inner function the array size is known.
  - The only time sizeof executes at runtime is when its applied to a dynamically allocated array

```
int *p = malloc(n);
sizeof(p); // WARNING: unknown at compile time
```

- `void foo( int p[] );`
  Evaluates to:
  `void foo( int *p );`
- The helper method only knows that it is being passed an int pointer, which is 4 bytes